

《数据结构》第二次实验报告

雷天乐 2025110528 25 数据科学 1 班

1 实验 1

实验任务：顺序表的基础操作：初始化，查找，插入，删除（索引从 1 开始）

实验时间：2026/09/16

知识点：顺序表用一段地址连续的存储单元依次存放数据元素，逻辑相邻的元素物理上也相邻，因此支持随机存取，按位查找的时间复杂度为 $O(1)$ 。其基本操作中，按值查找、插入和删除都需要移动或遍历元素，时间复杂度为 $O(n)$ ，顺序表通过结构体（基址、长度、容量）管理存储空间，容量不足时可动态扩容。它的优点是存取快、存储密度高，缺点是插入删除效率低、需要预先分配空间。

源代码

```
#include <stdio.h>
#include <stdlib.h>

#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10
#define OVERFLOW -2
#define ERROR 0
#define OK 1

typedef int ElemType;
typedef int Status;
typedef struct {
    ElemType *elem;
    int length;
    int listsize;
} SqList;

Status InitList_Sq(SqList* L){
```

```
L->elem = (ElemType*) malloc(LIST_INIT_SIZE * sizeof(ElemType));
if (!L->elem)
    return OVERFLOW; // 存储分配失败
L->length = 0; // 空表长度为0
L->listsize = LIST_INIT_SIZE; // 初始存储容量
return OK;
}

Status compare(ElemType a, ElemType b) { // + 必须在 main 之前
    return a == b;
}

Status LocateElem_Sq(SqList L, ElemType e, Status (*compare)(ElemType
, ElemType)) {
    int i = 1;
    ElemType *p = L.elem;
    while (i <= L.length && !(*compare)(*p++, e))
        i++;
    if (i <= L.length)
        return i;
    else
        return 0;
}

Status ListInsert_Sq(SqList *L, int i, ElemType e) {
    if (i < 1 || i > L->length + 1) return ERROR; // i 不合法
    if (L->length >= L->listsize) { // 空间已满, 扩容
        ElemType *newbase = (ElemType *)realloc(L->elem, (L->listsize
+ LISTINCREMENT) * sizeof(ElemType));
        if (!newbase) exit(OVERFLOW);
        L->elem = newbase;
        L->listsize += LISTINCREMENT;
    }
    ElemType *q = &(L->elem[i - 1]); // q 指向插入位置
    for (ElemType *p = &(L->elem[L->length - 1]); p >= q; --p)
        *(p + 1) = *p; // 右移元素
    *q = e;
    L->length++;
    return OK;
}
```

```
Status ListDelete_Sq(SqList *L, int i, ElemType *e) {
    if (i < 1 || i > L->length) return ERROR;           // i 不合法
    ElemType *p = &(L->elem[i - 1]);                     // p 指向被删元素
    *e = *p;
    ElemType *q = L->elem + L->length - 1;               // 表尾位置
    for (++p; p <= q; ++p)
        *(p - 1) = *p;                                   // 左移元素
    L->length--;
    return OK;
}

// 打印顺序表
void PrintList(SqList L) {
    printf("[");
    for (int i = 0; i < L.length; i++)
        printf("%d%s", L.elem[i], i == L.length - 1 ? "" : ", ");
    printf("]\n");
}

int main() {
    SqList L;
    InitList_Sq(&L);

    // 创建顺序表: 1, 2, 3, 4, 5, 6
    for (int i = 1; i <= 6; i++)
        ListInsert_Sq(&L, i, i);
    printf("创建后的表: ");
    PrintList(L);                                         // [1, 2, 3, 4, 5, 6]

    // 查找元素 4 和 8
    int pos = LocateElem_Sq(L, 4, compare);
    if (pos)
        printf("元素 4 存在, 在第 %d 个位置\n", pos);    // 在第 4 个位置
    else
        printf("元素 4 不存在\n");

    pos = LocateElem_Sq(L, 8, compare);
    if (pos)
```

```
        printf("元素 8 存在, 在第 %d 个位置\n", pos);
    else
        printf("元素 8 不存在\n"); // 走这分支

    // 在 i=4 处插入 0
    if (ListInsert_Sq(&L, 4, 0) == OK) {
        printf("插入后的表: ");
        PrintList(L); // [1, 2, 3, 0, 4, 5, 6]
    }

    // 删除 i=5 处的元素
    ElemType e;
    if (ListDelete_Sq(&L, 5, &e) == OK)
        printf("删除了第 5 个元素: %d\n", e); // 删掉了 4
    printf("删除后的表: ");
    PrintList(L); // [1, 2, 3, 0, 5, 6]

    free(L.elem);
    return 0;
}
```

输出结果

```
● PS D:\MinGW64> D:\MinGW64\test.exe
创建后的表: [1, 2, 3, 4, 5, 6]
元素 4 存在, 在第 4 个位置
元素 8 不存在
插入后的表: [1, 2, 3, 0, 4, 5, 6]
删除了第 5 个元素: 4
删除后的表: [1, 2, 3, 0, 5, 6]
○ PS D:\MinGW64>
```

图 1: 初始化, 查找, 插入, 删除

实验总结

通过本次实验，我掌握了顺序表的定义与实现方法，能够熟练运用顺序存储结构完成初始化、查找、插入和删除等基本操作。实验过程中，我理解了位序与数组下标之间的换算关系，掌握了通过函数指针实现按值查找的方法，并通过测试用例验证了各操作的正确性，加深了对时间复杂度和内存动态分配的理解，提高了 C 语言编程和调试能力。

2 实验 2

实验任务：单链表的查找，删除，插入（头插法，尾插法）（最小索引值为 1）

实验时间：2026/09/16

知识点：单链表是线性表的链式存储结构，每个结点由数据域和指针域组成，指针域指向后继结点，逻辑上相邻的元素物理位置不一定相邻，只能从头结点顺序访问，不支持随机存取。其插入和删除操作只需修改指针，无需移动元素，时间复杂度为 $O(1)$ ，而按位查找的时间复杂度为 $O(n)$ 。与顺序表相比，单链表不需要预先分配连续空间，插入删除效率更高，但存储密度低，不能随机访问，且找前驱结点较麻烦，这是其主要的不足之处。

源代码

```
#include <stdio.h>
#include <stdlib.h>

#define OK 1
#define ERROR 0

typedef int ElemType;
typedef int Status;

// 结点类型
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;

Status InitList_L(LinkList *L) {
    *L = (LinkList)malloc(sizeof(LNode));
    if (!(*L)) exit(0);
    (*L)->next = NULL;
    return OK;
}

// 2. 取值：取第 i 个元素，赋给 *e
Status GetElem_L(LinkList L, int i, ElemType *e) {
    LinkList p = L->next;    // p 指向第一个结点
```

```
    int j = 1;
    while (p && j < i) {
        p = p->next;
        ++j;
    }
    if (!p || j > i) return ERROR;    // 第 i 个元素不存在
    *e = p->data;
    return OK;
}

int LocateElem_L(LinkList L, ElemType e) {
    LinkList p = L->next;
    int i = 1;
    while (p && p->data != e) {
        p = p->next;
        i++;
    }
    if (p)
        return i;
    else
        return 0;
}

Status ListInsert_L(LinkList L, int i, ElemType e) {
    LinkList p = L;
    int j = 0;
    while (p && j < i - 1) {
        p = p->next;
        ++j;
    }
    if (!p || j > i - 1) return ERROR;
    LinkList s = (LinkList)malloc(sizeof(LNode));
    s->data = e;
    s->next = p->next;
    p->next = s;
    return OK;
}

// 5. 删除：删除第 i 个元素，用 *e 返回其值
Status ListDelete_L(LinkList L, int i, ElemType *e) {
```

```
    LinkList p = L;
    int j = 0;
    while (p->next && j < i - 1) {    // 找第 i-1 个结点
        p = p->next;
        ++j;
    }
    if (!(p->next) || j > i - 1) return ERROR;    // i 不合法
    LinkList q = p->next;                // q 指向被删结点
    p->next = q->next;                    // 摘下 q
    *e = q->data;
    free(q);
    return OK;
}

void CreateList_H(LinkList *L, int n) {
    *L = (LinkList)malloc(sizeof(LNode));
    (*L)->next = NULL;
    printf("请输入 %d 个整数: ", n);
    for (int i = 0; i < n; i++) {
        ElemType e;
        scanf("%d", &e);
        LinkList s = (LinkList)malloc(sizeof(LNode));
        s->data = e;
        s->next = (*L)->next;    // 插到头结点之后
        (*L)->next = s;
    }
}

// 7. 尾插法: 正位序输入 n 个元素建表
void CreateList_T(LinkList *L, int n) {
    *L = (LinkList)malloc(sizeof(LNode));
    LinkList r = *L;                // r 始终指向表尾
    printf("请输入 %d 个整数: ", n);
    for (int i = 0; i < n; i++) {
        ElemType e;
        scanf("%d", &e);
        LinkList s = (LinkList)malloc(sizeof(LNode));
        s->data = e;
```

```
        r->next = s;                // 挂到表尾
        r = s;                      // r 后移
    }
    r->next = NULL;                  // 表尾置空
}

void PrintList(LinkList L) {
    LinkList p = L->next;
    printf("[");
    while (p) {
        printf("%d", p->data);
        if (p->next) printf(", ");
        p = p->next;
    }
    printf("]\n");
}

int ListLength(LinkList L) {
    int len = 0;
    LinkList p = L->next;
    while (p) {
        len++;
        p = p->next;
    }
    return len;
}

// 销毁链表
void DestroyList(LinkList *L) {
    LinkList p = *L, q;
    while (p) {
        q = p->next;
        free(p);
        p = q;
    }
    *L = NULL;
}

int main() {
```

```
LinkList L;
ElemType e;
int pos;

// 尾插法建表：输入 1 2 3 4 5 6
CreateList_T(&L, 6);
printf("尾插法建表：");
PrintList(L);

// 头插法建表：输入 1 2 3 4
DestroyList(&L);
CreateList_H(&L, 4);
printf("头插法建表：");
PrintList(L);

// 取值：取第 3 个元素
if (GetElem_L(L, 3, &e) == OK)
    printf("第 3 个元素是： %d\n", e);

// 查找：找元素 3
pos = LocateElem_L(L, 3);
if (pos)
    printf("元素 3 在第 %d 个位置\n", pos);
else
    printf("元素 3 不存在\n");

// 插入：在第 2 个位置插入 99
if (ListInsert_L(L, 2, 99) == OK) {
    printf("插入 99 后：");
    PrintList(L);
}

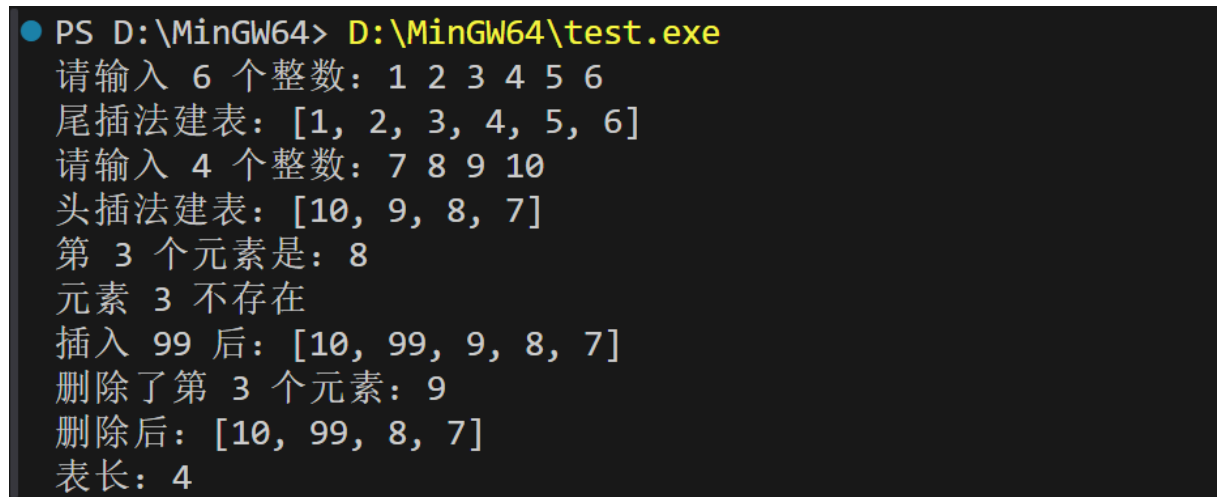
if (ListDelete_L(L, 3, &e) == OK)
    printf("删除了第 3 个元素： %d\n", e);
printf("删除后：");
PrintList(L);

printf("表长： %d\n", ListLength(L));

DestroyList(&L);
```

```
    return 0;  
}
```

输出结果



```
● PS D:\MinGW64> D:\MinGW64\test.exe  
请输入 6 个整数: 1 2 3 4 5 6  
尾插法建表: [1, 2, 3, 4, 5, 6]  
请输入 4 个整数: 7 8 9 10  
头插法建表: [10, 9, 8, 7]  
第 3 个元素是: 8  
元素 3 不存在  
插入 99 后: [10, 99, 9, 8, 7]  
删除了第 3 个元素: 9  
删除后: [10, 99, 8, 7]  
表长: 4
```

图 2: 头插法, 尾插法再建立单链表有所体现

实验总结

通过本次实验, 我掌握了单链表的存储结构和基本操作, 能够熟练实现初始化、查找、插入、删除以及头插法、尾插法建表等算法。实验加深了我对指针操作和结点链接关系的理解, 体会到链表插入删除无需移动元素的优势, 提高了 C 语言动态内存管理和程序调试能力。

3 实验 3

实验任务：循环链表解决约瑟夫问题

实验时间：2026/09/16

知识点：循环链表是首尾相接的链表，最后一个结点的指针指向头结点，可从任一结点出发遍历整个表，适合处理具有环形结构的问题。约瑟夫问题中，将 n 个人围成一圈，从第 s 个人开始报数，数到 m 的人出列，可用循环链表模拟：依次报数并删除对应结点，直到只剩一人。该方法直观体现了循环链表在解决报数、轮转类问题上的优势。

源代码

```
#include <stdio.h>
#include <stdlib.h>

typedef int ElemType;

typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;

// 创建不带头结点、只有尾指针的循环单链表，存入 1~n
void create_list(LinkList *Tail, int n) {
    LinkList p = NULL, s;
    for (int i = 1; i <= n; i++) {
        s = (LinkList)malloc(sizeof(LNode));
        s->data = i;
        if (p == NULL) {
            p = s;           // 第一个结点
            *Tail = s;
        } else {
            p->next = s;     // 接到尾部
            p = s;
        }
    }
    p->next = *Tail;        // 首尾相接，形成循环（此时 Tail 就是尾指针）
}

void Josephus(LinkList *Tail, int n, int m) {
```

```
LinkList p = (*Tail)->next, pre = *Tail; // p 从第 1 个人开始,
pre 指向其前驱
while (p->next != p) { // 多于一人时循环
    for (int i = 0; i < m - 1; i++) { // 报数, 走 m-1 步
        pre = p;
        p = p->next;
    }
    printf("出列的人是%d号\n", p->data);
    pre->next = p->next; // 摘下 p
    if (p == *Tail) // 若删的是尾结点, 更
        新尾指针
        *Tail = pre;
    free(p);
    p = pre->next; // 从下一个人继续报数
}

printf("优胜者是%d号\n", p->data);
free(p);
}

int main() {
    LinkList Tail;
    int n, m;
    printf("请输入总人数 n 和报数 m: ");
    scanf("%d %d", &n, &m);

    create_list(&Tail, n);
    Josephus(&Tail, n, m);

    return 0;
}
```

输出结果

```
● PS D:\MinGW64> D:\MinGW64\test.exe
请输入总人数 n 和报数 m: 8 3
出列的人是4号
出列的人是7号
出列的人是2号
出列的人是6号
出列的人是3号
出列的人是1号
出列的人是5号
优胜者是8号
○ PS D:\MinGW64> █
```

图 3: 8 人 123 报数报到 3 出列

实验总结

通过本次实验，我掌握了循环链表的结构特点及用尾指针管理循环单链表的方法，并成功用其模拟解决了约瑟夫问题。实验加深了我对指针操作、结点删除和循环遍历的理解，体会到循环链表在报数、轮转类问题中的应用优势，同时提高了 C 语言动态内存管理和程序调试能力。